

Experiment 'p_4_3_a_extract_uppercase' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_3_a_extract_uppercase

Natural language statement: Write a method that takes a string and returns only the uppercase letters in the string.

Method signature: p_4_3_a_extract_uppercase(s: string) returns (uppercase: string)

Ensures

- forall c :: c in uppercase ==> 'A' <= c <= 'Z'
- forall c :: c in s && 'A' <= c <= 'Z' ==> c in uppercase

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes a string and returns only the uppercase letters in the string.

The signature should be:

`p_4_3_a_extract_uppercase(s: string) returns (uppercase: string)`

The method should respect the following contract:

`ensures forall c :: c in uppercase ==> 'A' <= c <= 'Z', ensures forall c :: c in s && 'A'`
`↪ <= c <= 'Z' ==> c in uppercase`

Produce and show only the Dafny body of this method, including the curly braces that
`↪` surround it. Do not show the signature nor contract. You must not use 'assume' nor
`↪` 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dmei8sxeg7JxpAt1sT10fcFLSnwCm

Model: gpt-5.5-2026-04-23

Created at: 1780488608

Finish reason: stop

Usage: 328 tokens in, and 5510 tokens out

```
{
  uppercase := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall c :: c in uppercase ==> 'A' <= c <= 'Z'
    invariant forall c :: c in s[..i] && 'A' <= c <= 'Z' ==> c in uppercase
    decreases |s| - i
  {
    var ch := s[i];
    assert |s[..i+1]| == i + 1;

    if 'A' <= ch <= 'Z' {
      var oldUppercase := uppercase;
      assert forall c :: c in oldUppercase ==> 'A' <= c <= 'Z';
      assert forall c :: c in s[..i] && 'A' <= c <= 'Z' ==> c in oldUppercase;

      uppercase := oldUppercase + [ch];
      assert |uppercase| == |oldUppercase| + 1;
    }
  }
}
```

```

assert forall c | c in uppercase ensures 'A' <= c <= 'Z' {
  var j :| 0 <= j < |uppercase| && uppercase[j] == c;
  if j < |oldUppercase| {
    assert uppercase[j] == oldUppercase[j];
    assert oldUppercase[j] == c;
    assert c in oldUppercase;
  } else {
    assert j == |oldUppercase|;
    assert uppercase[j] == ch;
    assert c == ch;
  }
}

assert forall c | c in s[..i+1] && 'A' <= c <= 'Z' ensures c in uppercase {
  var j :| 0 <= j < |s[..i+1]| && (s[..i+1])[j] == c;
  if j < i {
    assert (s[..i+1])[j] == s[j];
    assert (s[..i])[j] == s[j];
    assert (s[..i])[j] == c;
    assert c in s[..i];
    assert c in oldUppercase;
    var k :| 0 <= k < |oldUppercase| && oldUppercase[k] == c;
    assert uppercase[k] == c;
    assert c in uppercase;
  } else {
    assert j == i;
    assert (s[..i+1])[j] == s[i];
    assert c == ch;
    assert uppercase[|oldUppercase|] == c;
    assert c in uppercase;
  }
}
} else {
  assert !('A' <= ch <= 'Z');
  assert forall c | c in s[..i+1] && 'A' <= c <= 'Z' ensures c in uppercase {
    var j :| 0 <= j < |s[..i+1]| && (s[..i+1])[j] == c;
    if j < i {
      assert (s[..i+1])[j] == s[j];
      assert (s[..i])[j] == s[j];
      assert (s[..i])[j] == c;
      assert c in s[..i];
    } else {
      assert j == i;
      assert (s[..i+1])[j] == s[i];
      assert c == ch;
      assert false;
    }
  }
}
}

i := i + 1;

```

```

}

assert s[..|s|] == s;
assert forall c | c in s && 'A' <= c <= 'Z' ensures c in uppercase {
  assert c in s[..|s|];
}
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_4_3_a_extract_uppercase/program.dfy(56,4): Error: invalid UnaryExpression
|
56 | } else {
| ~

problem_data/p_4_3_a_extract_uppercase/program.dfy(55,6): Error: expected either 'by' or
↪ a semicolon following the assert expression
|
55 | }
| ~

problem_data/p_4_3_a_extract_uppercase/program.dfy(55,6): Error: expected either 'by' or
↪ a semicolon following the assert expression
|
55 | }
| ~

problem_data/p_4_3_a_extract_uppercase/program.dfy(72,4): Error: invalid UnaryExpression
|
72 | }
| ~

problem_data/p_4_3_a_extract_uppercase/program.dfy(71,6): Error: expected either 'by' or
↪ a semicolon following the assert expression
|
71 | }
| ~

problem_data/p_4_3_a_extract_uppercase/program.dfy(81,0): Error: invalid UnaryExpression
|
81 | }
| ~

problem_data/p_4_3_a_extract_uppercase/program.dfy(80,2): Error: expected either 'by' or
↪ a semicolon following the assert expression
|
80 | }
| ~

```

7 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the

- ↪ Dafny body, including the curly braces that surround it. Do not show the signature
- ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmejnfMS8FrFtAxv56SsNpE2dav9D

Model: gpt-5.5-2026-04-23

Created at: 1780488711

Finish reason: stop

Usage: 1777 tokens in, and 4345 tokens out

```
{
  uppercase := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall c :: c in uppercase ==> 'A' <= c && c <= 'Z'
    invariant forall c :: c in s[..i] && 'A' <= c && c <= 'Z' ==> c in uppercase
    decreases |s| - i
  {
    var ch := s[i];
    assert s[..i + 1] == s[..i] + [ch];

    if 'A' <= ch && ch <= 'Z' {
      var prev := uppercase;
      assert forall c :: c in prev ==> 'A' <= c && c <= 'Z';
      assert forall c :: c in s[..i] && 'A' <= c && c <= 'Z' ==> c in prev;

      uppercase := prev + [ch];

      forall c | c in uppercase
        ensures 'A' <= c && c <= 'Z'
      {
        if c in prev {
          assert 'A' <= c && c <= 'Z';
        } else {
          assert c in [ch];
          assert c == ch;
        }
      }
    }

    forall c | c in s[..i + 1] && 'A' <= c && c <= 'Z'
      ensures c in uppercase
    {
      if c in s[..i] {
        assert c in prev;
        assert c in uppercase;
      }
    }
  }
}
```

```

    } else {
      assert c in [ch];
      assert c == ch;
      assert ch in uppercase;
    }
  }
} else {
  assert !('A' <= ch && ch <= 'Z');

  forall c | c in s[..i + 1] && 'A' <= c && c <= 'Z'
    ensures c in uppercase
  {
    if c in s[..i] {
      assert c in uppercase;
    } else {
      assert c in [ch];
      assert c == ch;
      assert 'A' <= ch && ch <= 'Z';
      assert false;
    }
  }
}

i := i + 1;
}

assert i == |s|;
assert s[..i] == s;

forall c | c in s && 'A' <= c && c <= 'Z'
  ensures c in uppercase
{
  assert c in s[..i];
}
}

```

Final Program

```

method p_4_3_a_extract_uppercase(s: string) returns (uppercase: string)
  ensures forall c :: c in uppercase ==> 'A' <= c <= 'Z'
  ensures forall c :: c in s && 'A' <= c <= 'Z' ==> c in uppercase
{
  uppercase := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall c :: c in uppercase ==> 'A' <= c && c <= 'Z'
    invariant forall c :: c in s[..i] && 'A' <= c && c <= 'Z' ==> c in uppercase
    decreases |s| - i
  {

```

```

var ch := s[i];
assert s[..i + 1] == s[..i] + [ch];

if 'A' <= ch && ch <= 'Z' {
  var prev := uppercase;
  assert forall c :: c in prev ==> 'A' <= c && c <= 'Z';
  assert forall c :: c in s[..i] && 'A' <= c && c <= 'Z' ==> c in prev;

  uppercase := prev + [ch];

  forall c | c in uppercase
    ensures 'A' <= c && c <= 'Z'
  {
    if c in prev {
      assert 'A' <= c && c <= 'Z';
    } else {
      assert c in [ch];
      assert c == ch;
    }
  }
}

forall c | c in s[..i + 1] && 'A' <= c && c <= 'Z'
  ensures c in uppercase
{
  if c in s[..i] {
    assert c in prev;
    assert c in uppercase;
  } else {
    assert c in [ch];
    assert c == ch;
    assert ch in uppercase;
  }
}
} else {
  assert !('A' <= ch && ch <= 'Z');

  forall c | c in s[..i + 1] && 'A' <= c && c <= 'Z'
    ensures c in uppercase
  {
    if c in s[..i] {
      assert c in uppercase;
    } else {
      assert c in [ch];
      assert c == ch;
      assert 'A' <= ch && ch <= 'Z';
      assert false;
    }
  }
}
}

i := i + 1;

```

```
}

assert i == |s|;
assert s[..i] == s;

forall c | c in s && 'A' <= c && c <= 'Z'
  ensures c in uppercase
{
  assert c in s[..i];
}
}
```

Total Token Usage

Input tokens: 2105

Output tokens: 9855

Reasoning tokens: 8312

Sum of 'total tokens': 11960

Experiment Timings

Overall Experiment started at 1780488608287, ended at 1780488792348, lasting 184061ms (184.06 seconds)

Iteration #1 started at 1780488608289, ended at 1780488711020, lasting 102731ms (102.73 seconds)

Iteration #2 started at 1780488711020, ended at 1780488792348, lasting 81328ms (81.33 seconds)